Programming Techniques 2026–2027

Lecture 9: Shared-Memory Parallelisation with OpenMP

Hannu Parviainen

Universidad de la Laguna

October 24, 2026

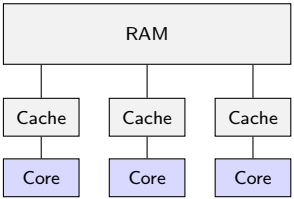
Introduction to OpenMP

What is OpenMP?

- OpenMP (Open Multi-Processing) is a portable programming interface for *shared-memory* parallel computing.
- It is a standard for Fortran and C/C++, and works on Linux, Windows, and macOS.
- It needs a compiler that supports OpenMP. Most modern compilers do, including `gfortran`.
- The first specification for Fortran came out in 1997. Version 6.0 was released in November 2024.
- OpenMP is built around a *fork–join* model: one thread runs the program and starts a team of threads where there is parallel work.
- A serial program can be parallelised in small steps, one loop at a time. There is no need to rewrite it from scratch.
- The serial and parallel versions of a program can live in the same source code.

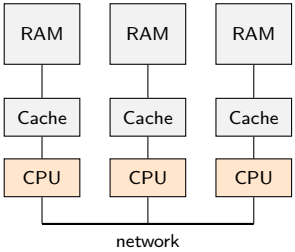
Shared and distributed memory

Shared memory: OpenMP



All the cores of one computer read and write the same memory. The threads of a program see the same variables.

Distributed memory: MPI



Each processor has its own memory. The processes exchange data by sending messages over a network.

OpenMP, MPI, or both?

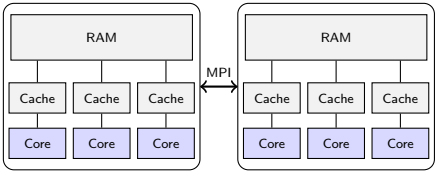
OpenMP

- runs on one shared-memory computer,
- parallelises a serial program step by step, in the same source,
- and leaves much of the work to the compiler.

MPI

- runs on shared- and distributed-memory systems,
- needs a program designed for it, and an MPI library,
- and leaves more of the work to the programmer.

Hybrid: MPI between nodes, OpenMP inside them



Large clusters combine the two: one MPI process per node, and OpenMP threads on the cores of that node.

Directives

- OpenMP is mostly written as *directives*. They tell the compiler how to share the work between threads. *Clauses* modify the directives.
- A directive is a comment that starts with the *sentinel* `!$omp`.
- A long directive is continued with `&`, and the next line starts again with `!$omp`.
- A compiler without OpenMP, or without the `-fopenmp` flag, sees the directives as ordinary comments and ignores them.
- So the same source compiles to a serial or a parallel program.

```
!$omp directive [clause, clause, ...]
...
!$omp end directive
```

A directive split over three lines:

```
!$omp parallel do private(x, y) &
!$omp shared(a, b) &
!$omp reduction(+:total)
```

Compiling with and without OpenMP:

```
gfortran -fopenmp -o prog prog.f90
gfortran -o prog prog.f90
```

Parallel Regions

A parallel region

- The `parallel` directive starts a team of threads.
- Every thread runs the code between `!$omp parallel` and `!$omp end parallel`.
- By default, there is one thread per logical core. Here, the environment variable `OMP_NUM_THREADS` sets the number to four.

```
program hello_omp
  implicit none

  print *, 'Before parallel'

  !$omp parallel
  print *, 'Inside parallel'
  !$omp end parallel

  print *, 'After parallel'
end program hello_omp
```

Compiled without `-fopenmp`:

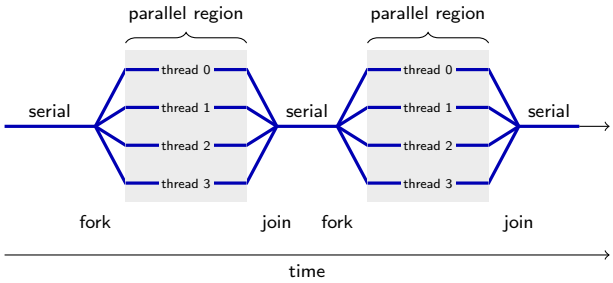
```
$ ./hello_omp
 Before parallel
 Inside parallel
 After parallel
```

Compiled with `-fopenmp`:

```
$ OMP_NUM_THREADS=4 ./hello_omp
 Before parallel
 Inside parallel
 Inside parallel
 Inside parallel
 Inside parallel
 After parallel
```

The fork–join model

- The program starts with one thread, the *master* thread (thread 0).
- At `!$omp parallel`, it *forks* a team of threads. The master thread is part of the team.
- At `!$omp end parallel`, the threads *join*: the master waits until all the threads have finished, and then continues alone.
- Starting and stopping a team costs time, so a parallel region should contain enough work.



Thread numbers: the omp_lib module

- The omp_lib module has procedures that query and control the threads:
  - omp_get_num_threads(): the number of threads in the team,
  - omp_get_thread_num(): the number of this thread, from 0 to $n-1$,
  - omp_get_wtime(): the wall-clock time in seconds.
- Each thread needs its own n_threads and id, so they are private. More about this later.
- The threads run at the same time, so the order of the output changes from run to run.

```
program thread_ids
  use omp_lib
  implicit none
  integer :: n_threads, id

  !$omp parallel private(n_threads, id)
  n_threads = omp_get_num_threads()
  id = omp_get_thread_num()
  print '(a, i0, a, i0)', 'Thread ', id, ' of ', n_threads
  !$omp end parallel
end program thread_ids
```

```
$ gfortran -fopenmp -o thread_ids thread_ids.f90
$ OMP_NUM_THREADS=4 ./thread_ids
Thread 2 of 4
Thread 1 of 4
Thread 3 of 4
Thread 0 of 4
```

Compiling without OpenMP

- The directives are ignored without `-fopenmp`, but the calls to `omp_lib` are ordinary Fortran. Compiling `thread_ids.f90` without OpenMP fails:

```
$ gfortran -o thread_ids thread_ids.f90
/usr/bin/ld: /tmp/ccM4ag4r.o: in function ‘MAIN__’:
thread_ids.f90:(.text+0xc): undefined reference to ‘omp_get_num_threads_’
/usr/bin/ld: thread_ids.f90:(.text+0x14): undefined reference to ‘omp_get_thread_num_’
collect2: error: ld returned 1 exit status
```

- The compiler finds the module, but the linker does not link the OpenMP runtime library that contains the procedures.
- The same happens with a compiler that does not support OpenMP. We would lose one of the main advantages of OpenMP: one source for both the serial and the parallel program.

Conditional compilation

- A line that starts with the sentinel `!$` followed by a space is compiled only with OpenMP. Otherwise, it is a comment.
- Use it for `use omp_lib` and for every call to an OpenMP procedure.
- The serial program gets the default values: one thread, number 0.
- Private variables start *undefined* in each thread, so the default values are set inside the parallel region.

```
program thread_ids_portable
  !$ use omp_lib
  implicit none
  integer :: n_threads, id

  !$omp parallel private(n_threads, id)
  n_threads = 1
  id = 0
  !$ n_threads = omp_get_num_threads()
  !$ id = omp_get_thread_num()
  print '(a, i0, a, i0)', 'Thread ', id, ' of ', n_threads
  !$omp end parallel
end program thread_ids_portable
```

```
$ gfortran -o thread_ids_portable thread_ids_portable.f90
$ ./thread_ids_portable
Thread 0 of 1
```

Setting the number of threads

There are three ways, from the weakest to the strongest:

1. the environment variable `OMP_NUM_THREADS` sets the default for the whole program,
2. `omp_set_num_threads(n)` sets it for the following parallel regions, and
3. the `num_threads(n)` clause sets it for one region.

- Without any of them, there is one thread per logical core.
- `omp_get_max_threads()` returns the size of the next team. Outside a parallel region, `omp_get_num_threads()` returns 1.

```
$ export OMP_NUM_THREADS=4   # this shell
$ ./prog
$ OMP_NUM_THREADS=2 ./prog   # one run
```

```
!$ call omp_set_num_threads(4)

!$omp parallel num_threads(2)
...
!$omp end parallel
```

Parallel Loops

Sharing the work of a loop

- Having every thread repeat the same work is pointless. A *worksharing* construct divides the work between the threads.
- `!$omp do` divides the iterations of the next `do` loop between the threads of the team. Each iteration is run by exactly one thread.
- `!$omp do` must be inside a parallel region. Otherwise, the team has one thread.
- The array `a` is shared. Each thread writes different elements of it.

```
program parallel_loop
  !$ use omp_lib
  implicit none
  integer, parameter :: n = 8
  real :: a(n)
  integer :: i, id

  !$omp parallel private(i, id) shared(a)
  id = 0
  !$ id = omp_get_thread_num()
  !$omp do
  do i = 1, n
    a(i) = i**2 + 3.0
    print '(a, i0, a, i0)', 'i = ', i, ' on thread ', id
  end do
  !$omp end do
  !$omp end parallel
  print '(*(f6.1))', a
end program parallel_loop
```

Sharing the work of a loop: the output

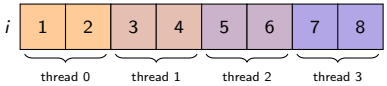
Without OpenMP:

```
i = 1 on thread 0
i = 2 on thread 0
i = 3 on thread 0
i = 4 on thread 0
i = 5 on thread 0
i = 6 on thread 0
i = 7 on thread 0
i = 8 on thread 0
```

Four threads:

```
i = 1 on thread 0
i = 2 on thread 0
i = 5 on thread 2
i = 6 on thread 2
i = 7 on thread 3
i = 8 on thread 3
i = 3 on thread 1
i = 4 on thread 1
```

- By default, each thread gets one contiguous block of iterations:



- The threads run at the same time, so the lines are printed in a different order every run.
- The array is the same in both cases:

```
4.0  7.0  12.0  19.0  28.0  39.0  52.0  67.0
```

The combined `parallel do`

- A parallel region that contains only one loop can be written with one combined directive.
- The clauses of both directives can be given to the combined one.
- The loop index is private automatically.
- There is an implicit *barrier* at `end do`: the threads wait for each other before they continue.
- The number of iterations must be known when the loop starts. A `do while` loop, or a loop with `exit`, cannot be shared.

```
!$omp parallel
!$omp do
do i = 1, n
  a(i) = i**2 + 3.0
end do
!$omp end do
!$omp end parallel
```

is the same as

```
!$omp parallel do
do i = 1, n
  a(i) = i**2 + 3.0
end do
!$omp end parallel do
```

Which loops can be parallelised?

- The iterations must be *independent*: they must give the same result when they run in any order, or at the same time.
- OpenMP does not check this. It does what the directive says, also when the result is wrong.
- A loop where each iteration needs the result of the previous one has to be rewritten, or left serial.

```
! Independent iterations: can be parallelised.
do i = 1, n
  c(i) = a(i) + b(i)
end do

! Each iteration needs the result of the previous
! one: cannot be parallelised as it is.
do i = 2, n
  a(i) = a(i-1) + b(i)
end do

! Every iteration updates the same variable:
! needs a reduction (later today).
do i = 1, n
  total = total + a(i)
end do
```

Nested loops and `collapse`

- `!$omp do` shares only the outer loop. Here, with four threads, one thread would be idle.
- `collapse(2)` shares all the $4 \times 3 = 12$ iterations.
- The loops must be perfectly nested.

Four threads:

```
(1,1) on thread 0
(2,1) on thread 0
(3,1) on thread 0
(4,1) on thread 1
(1,2) on thread 1
(2,2) on thread 1
```

```
(2,3) on thread 3
(3,3) on thread 3
(4,3) on thread 3
(3,2) on thread 2
(4,2) on thread 2
(1,3) on thread 2
```

```
program collapse
  !$ use omp_lib
  implicit none
  real :: a(4,3)
  integer :: i, j, id

  !$omp parallel private(i, j, id) shared(a)
  id = 0
  !$ id = omp_get_thread_num()
  !$omp do collapse(2)
  do j = 1, 3
    do i = 1, 4
      a(i,j) = i + j
      print '(3(a, i0))', '(', i, ',', j, ') on thread ', id
    end do
  end do
  !$omp end do
  !$omp end parallel
end program collapse
```

Shared and Private Data

Shared and private variables

- A `shared` variable exists once, and all the threads use the same memory.
- A `private` variable has a separate copy in each thread. The copies start *undefined*, and are lost at the end of the region.
- `firstprivate`: private, but each copy starts with the value from before the region.
- `lastprivate` (on `do`): private, and the value from the last iteration is copied out after the loop.
- By default, variables are shared, except the indices of parallel loops.
- `default(none)` turns the defaults off, so the compiler catches forgotten variables.

```
!$omp parallel do default(none) &
!$omp shared(a, b, c, n) private(x, y)
do i = 1, n
  x = a(i) + b(i)
  y = a(i) - b(i)
  c(i) = x * y
end do
!$omp end parallel do
```

Rules of thumb

Only read, or each thread writes its own elements: `shared`. A temporary value in an iteration: `private`. Updated by every iteration: `reduction`.

Race conditions

- A *race condition* happens when two threads access the same variable at the same time, and at least one of them writes to it.
- `total = total + 1` is three steps: read, add one, and write back. Two threads can read the same old value, and one addition is lost.
- The result is wrong, and different every run, without a crash or a warning. A program that gives the right answer once is not necessarily correct.

```
program race
  implicit none
  integer, parameter :: n = 1000000
  integer :: i, total

  total = 0
  !$omp parallel do shared(total)
  do i = 1, n
    total = total + 1
  end do
  !$omp end parallel do
  print '(2(a, i0))', 'total = ', total, ', expected ', n
end program race
```

```
$ OMP_NUM_THREADS=4 ./race
total = 258804, expected 1000000
$ OMP_NUM_THREADS=4 ./race
total = 276099, expected 1000000
```

Reductions

- A *reduction* combines many values into one: a sum, a product, a maximum...
- `reduction(op:list)` gives each thread a private copy of the variables, starting from the identity of op (0 for +, 1 for ∗). At the end, the copies are combined with op and the original value.
- `reduction(+:total)` fixes the race on the previous slide.
- The order of the additions changes, so a floating-point result can differ from the serial one in the last digits.

Operators: `+ * - .and. .or. .eqv. .neqv.`
Intrinsics: `max min iand ior ieor`

```
program reduction
  implicit none
  real :: a(5), b(5), dot
  integer :: i
  a = [0.1, 3.4, 7.4, 1.2, 8.3]
  b = [3.8, 0.1, 1.6, 0.1, 0.3]

  dot = 0.0
  !$omp parallel do reduction(+:dot)
  do i = 1, size(a)
    dot = dot + a(i) * b(i)
  end do
  !$omp end parallel do
  print '(a, f0.3)', 'a . b = ', dot
end program reduction
```

```
$ OMP_NUM_THREADS=4 ./reduction
a . b = 15.170
```

Summary: clauses for parallel loops

Clause	Meaning
`shared(list)`	one copy, used by all the threads
`private(list)`	one undefined copy per thread
`firstprivate(list)`	one copy per thread, starting with the value from before the region
`lastprivate(list)`	private, with the value of the last iteration copied out
`default(none)`	no defaults: every variable must be listed
`reduction(op:list)`	private copies, combined with op at the end
`collapse(n)`	share the iterations of n nested loops
`schedule(kind[, chunk])`	how the iterations are divided between the threads (Lecture 10)
`nowait`	on `end do`: no barrier at the end of the loop (Lecture 10)
`num_threads(n)`	on `parallel`: the size of the team

The force loop of course exercise 1 is a nested loop over the pairs of particles. Think about which variables are shared, which are private, and which are updated by many iterations.

Measuring time and speedup

- `omp_get_wtime()` returns the wall-clock time in seconds as a `real64`. Use the difference of two calls.
- Do not use `cpu_time` for parallel code: it adds up the time of all the threads.
- The *speedup* on p threads is

$$S(p) = \frac{T(1)}{T(p)},$$

  and the *parallel efficiency* is $E(p) = S(p)/p$.
- Time the same optimised build (e.g. `-O2`), repeat the runs, and do not print inside the timed part.

```
use omp_lib
use iso_fortran_env, only: real64
real(real64) :: t0, t1

t0 = omp_get_wtime()
!$omp parallel do
do i = 1, n
  b(i) = sin(a(i)) + 2*a(i)
end do
!$omp end parallel do
t1 = omp_get_wtime()

print '(a, f6.3, a)', 'Time: ', t1 - t0, ' s'
```

```
$ OMP_NUM_THREADS=1 ./prog
$ OMP_NUM_THREADS=4 ./prog
```

Exercise 1: parallel matrix product

Tasks

1. Write a subroutine that computes the matrix product $C = AB$ of two $n \times n$ matrices with three nested loops, as in the matrix product exercise of Lecture 3.
2. Write a second version, parallelised with OpenMP so that different threads compute different columns of C. Which variables are shared and which are private? Use `default(none)`.
3. Write a program that fills two matrices with random numbers, multiplies them with both versions and with the `matmul` intrinsic, checks that the results agree, and prints the wall time each one took. Use `omp_get_wtime`.
4. Run the program with $n = 200$ and $n = 1000$, and with 1, 2, 4, and 8 threads. How much faster is the parallel version? Does the program still compile and run without `-fopenmp`?

Exercise 2: the trapezoidal rule

Tasks

1. Write a program that computes the integral

$$I = \int_0^3 x \sin x \, dx$$

   with the trapezoidal rule and n intervals, using a parallel `do` loop and a reduction. Use 64-bit reals.
2. Compare the result with the exact value $I = \sin 3 - 3\cos 3 \approx 3.1110$. How does the error change when n is doubled?
3. Remove the `reduction` clause. What happens to the result, with one thread and with several?

Exercise 3: the Wallis product

Tasks

1. The Wallis product gives π as an infinite product:

$$\frac{\pi}{2} = \prod_{n=1}^{\infty} \frac{4n^2}{4n^2 - 1} = \frac{2}{1} \cdot \frac{2}{3} \cdot \frac{4}{3} \cdot \frac{4}{5} \cdot \frac{6}{5} \cdot \frac{6}{7} \cdots$$

   Write a program that computes π from the first N terms with a parallel reduction.
2. How many terms are needed to get five correct digits, 3.1415? Is the answer the same with 32-bit and 64-bit reals?
3. Does the result depend on the number of threads? Why?